Olingo: **Threshold Lattice Signatures with DKG and Identifiable Abort**

Kamil Doruk Gur
University of Maryland
College Park, USA
dgur1@proton.me

Patrick Hough
Universität der Bundeswehr München
Munich, Germany
patrick.hough@unibw.de

Jonathan Katz
Google
Washington DC, USA
jkatz2@gmail.com

Caroline Sandsbråten
Norwegian University of Science and Technology
Trondheim, Norway
caroline.sandsbraten@ntnu.no

Tjerand Silde
Norwegian University of Science and Technology
Trondheim, Norway
tjerand.silde@ntnu.no

A ARTIFACT APPENDIX

A.1 Abstract

The artifact is a collection of software used to benchmark Olingo. It includes a C implementation for the passively secure threshold signature scheme and the passively secure distributed key-generation protocol, a C++ implementation of the linearity proofs, Python/LaZer implementations of the LNP proofs, and scripts for estimating the security of the concrete parameter sets used in the paper and computing the communication sizes. The artifact is packaged as a Git repository, with individual components organized into separate subdirectories along with Makefiles to build each component.

When built and executed, the artifact runs computational benchmarks for Olingo's key-generation, signing, verification, and zero-knowledge-proof components. It outputs execution times for the selected number of parties and thresholds. The implementation simulates the computation of a single party where appropriate and does not measure network or broadcast-channel latency. The artifact also outputs communication sizes.

The artifact is licenced under the MIT Licence as specified in the root-level `LICENCE` file. Third-party components included in or used by the artifact remain subject to their respective licenses.

A.2 Description & Requirements

The artifact is organized as a collection of implementations and benchmark code for the main computational components of Olingo. A top-level `README.md` provides an overview of the repository and basic build and execution instructions. The main directories are:

- `olingo_passive/`: C source code for benchmarking the passive Olingo scheme. The `ref/` subdirectory contains the implementation, Makefiles, and benchmark targets for distributed key generation, signing-key generation, threshold signing and verification. The accompanying `build.sh` script builds the passive benchmark targets for a selected number of users and threshold.
- `lin_proofs__bdlop/`: C++ source code and benchmarks for the BDLOP commitments and the linearity proofs used by the active security components of Olingo. This component includes NFLlib as a dependency and provides a Makefile target for building the `olingo` benchmark executable.
- `olingo__lazer/`: Python and LaZer benchmark code for the LNP proofs used in the paper. The individual proof benchmarks are located under the `python/` subdirectory and are built using their respective Makefiles. The documentation for the LaZer library can be built from the subdirectory `docs/`.
- `lattice-estimator/`: Script and supporting files for estimating the concrete security of the Olingo parameter sets using the lattice estimator https://lattice-estimator.readthedocs.io/en/latest/ in addition to communication cost calculations.

Benchmark inputs are generated by the implementations using the 128-bit security parameter set and are built using the `USERS` and `THRESHOLD` build variables. Where appropriate, the benchmarks simulate the execution of one party and use dummy objects representing inputs produced by the other parties, to model the computational cost of processing real protocol inputs.

A.2.1 Security, privacy, and ethical concerns. The artifact processes no personal, confidential, or externally supplied data, and its execution raises no known privacy or ethical concerns. It does not require elevated privileges or modify files outside its source and build directories.

The artifact is an academic research prototype intended only for benchmarking and parameter evaluation, and is therefore not a production-ready cryptographic library. Parts of the implementation are not constant-time, and the code has not passed a dedicated security audit.

Some of the larger distributed key-generation and proof benchmarks can consume substantial memory and CPU time. The benchmark programs themselves do not communicate with external services or require network access during execution.

A.2.2 How to access. The artifact associated with the Olingo paper is available through Zenodo at:

https://zenodo.org/records/21822091

The artifact is also permanently preserved by Software Heritage and can be accessed at:

https://archive.softwareheritage.org/browse/origin/directory/?origin_url=https://github.com/carrosa/Olingo_Benchmarks

The exact archived revision is identified by the following SWHID:

`swh:1:dir:192bff31541e950eede25624efd542deb9ea53e1;`
`origin=https://github.com/carrosa/Olingo_Benchmarks;`
`visit=swh:1:snp:8ce610ff48c5fb2a7d9fe49c5f2b29b8496d1eea;`
`anchor=swh:1:rev:10b2137665b79bce92d54a6ecac7903ae86ed2ec`

The maintained repository is available at:

https://github.com/carrosa/Olingo_Benchmarks

The Zenodo archive provides the fixed version associated with this paper, whereas the GitHub repository may receive later documentation updates and compatibility fixes.

When the artifact is downloaded from Software Heritage, the extracted top-level directory may have a name containing special characters. This can interfere with the NFLlib build process. If necessary, rename the extracted top-level directory to a conventional name, such as `Olingo_Benchmarks`, before building the artifact.

A.2.3 Hardware dependencies. The build configuration is intended for a recent x86-64 processor.

The passive signing benchmark is compiled with the `-msse4.1` and `-maes` compiler options. Consequently, the processor used to execute this benchmark must support the SSE4.1 and AES instruction-set extensions. The remaining passive benchmarks are compiled with `-march=native`, which allows the compiler to use instructions supported by the build machine. Executables produced with this option should therefore be executed on the same machine, or on a machine supporting the same instruction-set extensions.

The performance measurements reported in the paper were obtained on a desktop machine with an Intel Core i9-14900 CPU and 64 GB of RAM. This exact processor and memory configuration are not required for functional evaluation. Small sanity-check configurations can be run on a typical recent x86-64 Linux machine, although execution times will differ.

At least 8 GB of RAM is recommended for building and running small benchmark configurations, but no strict memory requirement has been established. The larger distributed key-generation benchmarks can require substantially more memory and runtime. A machine with approximately 64 GB of RAM is recommended when attempting to reproduce the largest configurations reported in the paper as that was the RAM used during benchmarking.

The benchmarks do not require a particular number of CPU cores. They primarily measure the computation of one simulated party, and computations performed by different protocol parties are not executed as a distributed workload.

A.2.4 Software dependencies. The artifact is intended for a 64-bit Linux environment. The measurements in the paper were obtained using Ubuntu 24.04.2. Other recent Linux distributions may work but have not necessarily been tested. The software requirements are:

- a C compiler and a C++ compiler supporting the options used by the supplied Makefiles.
- GNU Make and standard Unix build utilities.
- Git, if the artifact is obtained by cloning a repository.
- CMake, for configuring and building NFLlib.
- GMP, for multiple-precision integer arithmetic.
- MPFR, for multiple-precision floating-point arithmetic.
- FLINT 2.9, or a compatible version.
- Python 3 and the Python packages required by the LaZer benchmarks.
- the LaZer library and its build dependencies, as documented in the corresponding artifact subdirectory.
- SageMath and NumPy for regenerating the NTT zeta values and related parameter files.
- Python/Sage dependencies of the lattice estimator for running the parameter-security estimates.

The artifact includes the source code for NFLlib, LaZer, and the version of the lattice estimator used to produce the results in the paper. These components do not need to be downloaded separately, but NFLlib and LaZer must be compiled before running their corresponding benchmarks.

No proprietary software is required. Exact package names differ between Linux distributions. The top-level `README.md` and the README files in the individual component directories provide component-specific installation and build instructions.

A.2.5 Benchmarks. The artifact includes source code for building and executing computational benchmarks cited in the Olingo paper. The artifact does not require any external datasets, models or workloads. The benchmarks cover the following components:

- Passive distributed key generation (DKG).
- Threshold signing and verification.
- Linearity proof generation and verification.
- LNP proof generation and verification.
- Security estimation for the concrete parameter set for 128-bit computational security given in Olingo.
- Computation of communication sizes.

The passive benchmarks are parameterized by the number of parties and the threshold using the `USERS` and `THRESHOLD` Make variables. They simulate the local computation of one party and where needed, use representative dummy inputs for the remaining parties. Computations that would be performed independently by different parties are treated as parallel, and network latency and broadcast-channel costs are excluded.

The paper reports results obtained using an Intel Core i9-14900 CPU with 64 GB of RAM running Ubuntu 24.04.2. Timings for signing, verification, distributed key generation, and the zero-knowledge proof components were averaged over multiple executions, with an exception for the largest key-generation configuration. Exact timing results are not expected to match exactly across machines because they depend on the processor, memory, and system load.

A.3 Set Up

The artifact is intended for a recent 64-bit Linux system and was tested on Ubuntu 24.04.2. The setup consists of obtaining the repository, installing the required build and mathematical libraries, building the included dependencies, and compiling a small benchmark instance. Detailed and component-specific instructions are provided in the top-level `README.md` and in the README files of the individual artifact directories.

No system-wide installation of Olingo is required. The source code, dependencies included with the artifact, and generated benchmark executables remain within the artifact directory with the exception of commonly used libraries.

A.3.1 Installation.

Obtaining the artifact. Download and extract the artifact from Zenodo:

https://zenodo.org/records/21822091.

Or from the preserved version on Software Heritage:

https://archive.softwareheritage.org/browse/origin/directory/?origin_url=https://github.com/carrosa/Olingo_Benchmarks.

Alternatively, after the anonymous review period, clone the maintained repository:

```
git clone https://github.com/carrosa/Olingo_Benchmarks.git
cd Olingo_Benchmarks
```

System dependencies. Install a C compiler, a C++ compiler, GNU Make, CMake, Git, Python 3, and the development packages for GMP, MPFR, and FLINT. The implementation was tested with FLINT 2.9. SageMath and NumPy are additionally required for regenerating the NTT constants and running the corresponding parameter scripts.

On Ubuntu and other Debian-based systems, most of the system dependencies can be installed using the distribution package manager.

Passive Olingo implementation. The passive implementation is located under `olingo_passive/ref/`. Once the system dependencies are installed, no additional configuration is required. Benchmark executables are generated locally by the supplied Makefile or `build.sh` script for the passive security benchmarks. The parameters `USERS` and `THRESHOLD` select the total number of parties and the threshold, respectively.

The supplied `build.sh` script can be used to build all passive benchmark targets for one parameter configuration:

```
cd olingo_passive/ref
bash build.sh --users=<int> --threshold=<int>
```

This produces the executables `test/dkg_<n>_<t>`, `test/preenc_<n>_<t>`, `test/encrypt_<n>_<t>`, `test/as2_<n>_<t>`, `test/ghkss256_<n>_<t>`, which corresponds to the passive $\mathsf{KGen}_{\mathcal{E}}$, the first, second and third round of $\mathsf{KGen}_{\mathcal{S}}$ and the Olingo signature scheme respectively.

Linearity-proof implementation. The linearity-proof benchmarks depend on NFLlib. NFLlib is included with the artifact but must be configured and built before compiling the benchmark. From the `<artifact-root>/lin_proofs__bdlop` directory, run:

```
mkdir -p deps
cd deps
cmake ../NFLlib \
 -DCMAKE_BUILD_TYPE=Release \
 -DNFL_OPTIMIZED=ON
make
make test
cd ..
make olingo
```

A successful build produces the `olingo` benchmark executable.

LaZer proof benchmarks. Starting from the root directory of the artifact, first build the bundled LaZer library:

```
cd olingo__lazer
make
```

Next, build the LaZer Python components:

```
cd python
make all
```

Finally, build each of the four proof benchmark modules:

```
cd bnd_E_prf
make clean
make

cd ../dkg_prf1
make clean
make

cd ../pi_r
make clean
make

cd ../pi_si
make clean
make
```

After compiling the modules, run each benchmark by executing `ctxr.py` from the corresponding component directory. The `python` command executing `ctxr.py` must refer to the Python environment in which the SageMath dependency is installed.

Security-estimation scripts. The scripts under `lattice-estimator/` package and use the lattice estimator and its SageMath dependencies. These dependencies are required only for reproducing the parameter-security estimates and are independent of the C and C++ timing benchmarks. This script also calculates the communication sizes listed in the main paper. The security estimation script used in the Olingo paper is named `sigparams.sage`.

A.3.2 Basic test.

A small passive distributed-key-generation instance provides a functionality test for the C implementation and its dependencies. From the top-level artifact directory, run:

```
cd olingo_passive/ref
make bench USERS=3 THRESHOLD=2 DKG=1
```

A successful build produces:

```
test/dkg_3_2
```

The build should terminate without compiler or linker errors and produce a benchmark executable under `test/`, with a name containing the selected values of `USERS` and `THRESHOLD`. Run the generated executable using:

```
./test/dkg_3_2
```

Successful execution prints timing information for the DKG and terminates normally. Exact timing values depend on the processor, compiler, frequency scaling, and current system load. The functionality test is successful if:

- The source files compile without errors.
- The expected executable is produced.
- The executable terminates without errors.

The output has the following form:

```
BENCH: DKG round 1 = ... cycles (... seconds)
```

```
BENCH: DKG round 2 = ... cycles (... seconds)
BENCH: DKG round 3 = ... cycles (... seconds)
```

The threshold-signing implementation can be checked separately with:

```
make ghkss USERS=3 THRESHOLD=2
```

With output on the form:

```
as_sign_round1: ... seconds
as_sign_round1: ... ms
as_sign_round2: ... seconds
as_sign_round2: ... ms
as_sign_round3: ... seconds
as_sign_round3: ... ms
as_sign_comb: ... seconds
as_sign_comb: ... ms
as_sign_verify: ... seconds
as_sign_verify: ... ms
```

For the linearity-proof component, run the following command after NFLlib has been built:

```
make olingo
./olingo
```

Successful execution prints timing measurements the linearity-proofs and terminates normally. These tests confirm basic functionality only. The experiments used to validate the paper's performance claims are described in the Evaluation Workflow below.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1): Olingo provides passive benchmarks for distributed key generation, threshold signing, combination, and verification, parameterized by the number of parties and the threshold. Experiment (E1) validates this claim and supports the implementation results described in Section 7.2 and Tables 5 and 7.

(C2): The distributed key-generation cost consists of the passive $\mathsf{KGen}_{\mathcal{E}}$ and $\mathsf{KGen}_{\mathcal{S}}$ computations together with the LaZer norm-bound proofs and the BDLOP linearity proofs required for active security. Experiment (E2) validates the individual components and explains their mapping to the distributed key-generation results in Table 5.

(C3): The active-security signing timings in Table 7 can be reconstructed by combining the passive signing measurements with the independently benchmarked proof costs. The second-round timing includes the LaZer proof π_r, and the third-round timing includes the LaZer norm-bound proof and BDLOP linearity proof that together implement π_{ds_i}. Signature combination and verification are measured directly by the passive signing benchmark. Experiment (E3) validates this claim.

(C4): The parameter script reproduces the Level-I public-key size, signature size, active signing communication, and hardness estimation reported in the paper for $(n, t) = (1024, 1023)$. Experiment (E4) validates this claim and corresponds to Section 7.1 and Tables 3, 4, and 6.

A.4.2 Experiments.

(E1): **Passive functionality** [10–20 person-minutes + approximately 1–10 compute-minutes for small instances; larger configurations may require several hours and substantial memory for multiple runs of the passive DKG].
This experiment validates Claim (C1). It exercises the parameterized passive DKG and signing implementation. Small configurations are sufficient for functional evaluation. Reproduction of the largest DKG configurations should be treated as an optional full-scale experiment.
Preparation: Compile the passive implementation dependencies as described above and enter:

```
cd olingo_passive/ref
```

Select values for `USERS` and `THRESHOLD`. For the DKG configurations reported in Table 5, the full threshold configurations use:

```
THRESHOLD=USERS-1
```

Execution: First, compile the small functionality configurations:

```
bash build.sh --users=3 --threshold=2
```

To execute the executables produced by the compiler:

```
./test/dkg_3_2
./test/ghkss256_3_2
./test/preenc_3_2
./test/encrypt_3_2
./test/as2_3_2
```

The parameter values are reflected in the executable names. In this example, both executables use `USERS` = 3 and `THRESHOLD` = 2.
To scale to larger thresholds and more users, repeat the relevant benchmarks with increasing `USERS` and `THRESHOLD`. The paper reports DKG measurements for $n \in \{16, 32, 64, 128\}$ with $t = n - 1$, and signing measurements for thresholds including 4, 16, 64, 256, and 1023.
Results: The executable `test/dkg_3_2` benchmarks the three rounds of the passive distributed key-generation protocol. It prints the CPU-cycle count and elapsed time for each round. One execution with `USERS` = 3 and `THRESHOLD` = 2 produced:

```
BENCH: DKG round 1 = 134305787 cycles (0.436271 seconds)
BENCH: DKG round 2 = 520528287 cycles (1.738665 seconds)
BENCH: DKG round 3 = 2283672 cycles (0.023955 seconds)
```

The executable `test/ghkss256_3_2` runs the passive signing implementation. This outputs the execution times of the three passive signing rounds, signature combination, and signature verification. One execution with `USERS` = 3 and `THRESHOLD` = 2 produced:

```
as_sign_round1: 2.246753 ms
as_sign_round2: 40.155627 ms
as_sign_round3: 4.800516 ms
as_sign_comb: 1.347444 ms
as_sign_verify: 1.893087 ms
```

To benchmark other configurations, replace `USERS` and `THRESHOLD` in the build commands and execute the correspondingly named binaries. For example:

```
bash build.sh --users=16 --threshold=15

./test/dkg_16_15
./test/ghkss256_16_15
./test/preenc_16_15
./test/encrypt_16_15
./test/as2_16_15
```

For the full-threshold DKG configurations used in Table 5, set `THRESHOLD` = `USERS` − 1. The paper reports results for $n \in \{16, 32, 64, 128\}$ with $t = n - 1$. The passive results from this experiment form the base protocol costs used in experiments (E2) and (E3), where the corresponding active-security proof costs are added.

(E2): **Distributed key-generation** [30–60 person-minutes + approximately 5-30 compute-minutes for reduced configurations; several compute-hours to reproduce all benchmarks].
This experiment validates Claim (C2) and explains how the artifact components map to Table 5. Table 5 is assembled from passive key-generation timings and the associated active-security proofs. It is not generated by one executable.
Preparation: Build the passive implementation, all LaZer modules, and the `lin_proofs__bdlop` benchmark as described above.
Execution: Run the passive KGenE and KGenS benchmarks using $t = n - 1$ for the desired values of n.

- `dkg_prf1`, corresponding to the norm-bound proof in π_{KGenE}.
- `pi_si`, corresponding to π_{s_i};
- DKG linearity proof 2, corresponding to the codeword proof.
- DKG linearity proof 3, corresponding to the Lagrange reconstruction proof.

```
cd <artifact-root>/olingo_passive/ref

bash build.sh --users=<n> --threshold=<t>

# Then run using:
./test/dkg_<n>_<t> # KGenE
./test/preenc_<n>_<t> # KGenS round 1
./test/encrypt_<n>_<t> # KGenS round 2
./test/as2_<n>_<t> # KGenS round 3
```

For Table 5, use $n \in \{16, 32, 64, 128\}$ and $t = n - 1$.
Run the LaZer proof benchmarks from their respective directories (see how to build lazer above):

```
cd <artifact-root>/olingo__lazer/python/dkg_prf1
  python ctxr.py

  cd ../pi_si
  python ctxr.py
```

Finally, run the linearity-proof benchmark:

```
cd <artifact-root>/lin_proofs__bdlop
./olingo
```

Record the generation and verification timings labelled as DKG linearity proof 2 and DKG linearity proof 3.
Results: Construct the Table 5 key-generation costs from the passive KGenE and KGenS timing and the proof-generation timings associated with that protocol.
The reported linearity-proof benchmark measures one proof execution. For the paper's parameter dimensions, its contribution to the complete protocol is obtained the following way:

- Multiply the DKG codeword-proof time by 21.
- Multiply the DKG Lagrange-proof time by $21 \cdot 15$.

The resulting proof costs are combined with the relevant passive timing and the LaZer proof timings. Minor differences are expected because the LaZer Python wrapper and the underlying CPU affect execution time.
The paper averages most timing measurements over ten executions. For a faster functional evaluation, one execution of each component is sufficient. Repeating each measurement ten times gives a comparison closer to the methodology used in Section 7.2.

(E3): **Signing** [10–30 person-minutes + approximately 5–30 compute minutes].
This experiment validates Claim (C3) and maps the passive and proof benchmark outputs to the signing results in Table 7.
Preparation: Build the passive signing benchmark, the `pi_r` and `bnd_E_prf` LaZer modules, and the C++ linearity-proof benchmark like described above.
Execution: Run the passive signing benchmark for one or more thresholds. The benchmarks to run are the ones produced by the following executables: `olingo_passive/ref/test/ghkss256_<n>_<t>`, `olingo__lazer/python/pi_r/ctxr.py`, `olingo__lazer/python/bnd_E_prf/ctxr.py`, `lin_proofs__bdlop/olingo`.

```
cd <artifact-root>/olingo_passive/ref
make ghkss USERS=<n> THRESHOLD=<t>
./test/ghkss256_<n>_<t>
```

Run the LaZer proof benchmarks for π_r and the norm-bound component of π_{ds_i}:

```
cd <artifact-root>/olingo__lazer/python/pi_r
make
python ctxr.py
cd <artifact-root>/olingo__lazer/python/bnd_E_prf
make
python ctxr.py
```

Run the `olingo` C++ benchmark and record the result labelled as the signing-round-3 linearity proof:

```
cd <artifact-root>/lin_proofs__bdlop
./olingo
```

From the olingo executable output, record the proof-generation and verification timings labelled as the signing-round-3 linearity proof.
Results: Map the measurements to Table 7 as follows:

- $\text{Sign}S_1$ is the passive first passive signing round only.
- $\text{Sign}S_2$ is the passive second signing round together with proof π_r.

- SignS_3 is the passive third signing round together with proof π_{ds_i}.
- π_{ds_i} consists of the LaZer `bnd_E_prf` norm-bound proof and the signing-round-3 linearity proof from `lin_proofs__bdlop`.
- `Comb` and `Verify` are taken from the passive combination and verification measurements.

For the second signing stage, account separately for proof generation and proof verification. Verification of the proofs received from other participating signers is threshold-dependent and should be scaled by the number of proofs verified in the selected configuration.

(E4): **Parameter sizes and communication** [5 person-minutes + approximately 1–5 compute-minutes].
This experiment validates Claim (C4). It provides the most direct numerical reproduction of the headline size and communication results in the paper.
Preparation: Install SageMath and enter the `lattice-estimator/` directory. The exact estimator revision used for the paper is included with the artifact, and no upstream estimator checkout is needed.
Confirm that `sigparams.sage` is configured for:

```
(n, t) = (1024, 1023)
```

Execution: Run:

```
cd lattice-estimator
sage sigparams.sage
```

The script performs the parameter calculations, checks the required correctness constraints, invokes the lattice estimator for the relevant LWE and SIS instances, and computes the resulting key, signature, and communication sizes.
Parameters are set to optimize the combined public key + signature size. The script does however allow one to experiment with other configurations by changing the `balanced` variable.
Correctness conditions are ensured by setting dependent variables in an automatic way, given pre-determined independent variable (e.g. threshold sizes, ring dimension etc.). However, security guarantees must be ensured with a somewhat more trial-and-improvement approach. As is often the case in lattice constructions, the sheer number of parameters and associated dependency relationships can make it near-impossible to choose their values in a systematic way to ensure both correctness and security[1]. As such, we have attempted to automate all choices which can be optimized but the user wishing to experiment with other parameter sets may need to make a few attempts to achieve the desired security level across all assumptions. We note that the major levers to pull are $k, \ell, \sigma_y, \sigma_w$.
By line 161, all parameters defining the basic signature scheme are set. Lines 162 - 206 concern the security of the basic signature scheme where the hardness of assumptions underlying the partial signing key, partial commitment, and verification key are estimated.
Lines 208 - 258 set the parameters of the encryption layer. Note that `expansionfactor` and `adjust_factor` simply exist to help set $\hat{k}$ and $\hat{\ell}$ more easily given k and ℓ. `expansionfactor` is used as a multiplicative scale-up factor where as `adjust_factor` is an additive factor for fine tuning. Lines 262 - 305 estimate the security of ciphertext elements v and u as well as the encryption public key pk and the BDLOP commitment.
Lines 310 - 450 compute the sizes or all relevant objects and communication costs. All sizes are derived from the parameters computed above with the exception of LNP zero-knowledge proofs whose sizes are computed by the LaZeR library externally and simply added manually for our 128-bit secure parameters. Note, readers wishing to recompute active key generation and signing sizes for different parameters will need to re-run the LNP proofs using their newly-computed parameter sets to obtain new proof sizes to be imported.
Results: For the Level-I parameter set, the expected output includes values parameters targeting 128-bits of security for all relevant hardness assumptions, and the expected output of communication cost is:

```
    SIG size in KB: 11.30
    PK size in KB: 4.423
    DKG_E active com cost = 2003.52 MB
    DKG_S total active cost = 2003.80 MB
  Passive signing communication cost = 242.04 KB
    Active signing cost = 852.30 KB
    Online signing cost = 563.80 KB
    Optimistic signing cost = 76.73 KB
```

A.5 Version

Based on the LaTeX template for Artifact Evaluation V20220926.

[1] we refer the reader to the LaZeR implementation which runs many rounds of 'guess and tweak' trials before proceeding with parameters that are 'good enough'.